

AUDIT COMPLET D'APPLICATION — LUDEX RATING COMPANION

Executive Summary

Ludex Rating Companion est une application web progressive (PWA) monopage de notation et de suivi de films et séries, fonctionnant majoritairement côté client avec un backend serverless minimaliste. Le choix d'une architecture Vanilla JS sans framework moderne est un parti pris fort. L'application est rapide, fluide et possède une direction artistique assumée. Toutefois, cette absence d'abstraction structurelle entraîne des limites en termes de maintenabilité, d'extensibilité et de testabilité des composants UI. La qualité perçue de l'interface est "Premium", mais le code sous-jacent commence à souffrir de sa taille.

Compréhension de l'application

- **Type d'application** : Progressive Web App (PWA) / Single Page Application (SPA).
- **Objectif fonctionnel** : Noter, tracker des films/séries, découvrir de nouvelles œuvres, consulter l'historique et des statistiques personnelles.
- **Utilisateurs cibles** : Cinéphiles, sériophiles, utilisateurs réguliers cherchant une alternative personnalisable à Letterboxd ou Trakt.
- **Stack technique** : Vanilla JavaScript (ES6+), HTML5, CSS3.
- **Backend** : Fonctions Serverless Vercel (`api/search.js` , `api/sync.js`).
- **Dépendances externes** : API TMDb (données films/séries), OMDb (notes externes), Supabase (synchronisation des données utilisateur).
- **Architecture générale** : Frontend statique généré par concaténation de fichiers JS modulaires (`src/*.js` -> `app.js`). L'état est géré globalement et persisté dans le `localStorage` .
- **Flux de données** : API TMDb -> Serverless Proxy (mise en cache) -> Frontend -> `LocalStorage` <-> Supabase (sauvegarde asynchrone par `sync_code`).

Architecture

Score : 6/10

L'architecture est basique mais fonctionnelle. La séparation en multiples fichiers dans `src/` numérotés pour garantir l'ordre d'exécution est un palliatif à l'absence d'un vrai bundler (comme Vite ou Webpack) ou de modules ES.

Points forts :

- Isolation de la logique métier pure dans `03b-pure-logic.js` , ce qui la rend parfaitement testable unitairement en Node.
- Proxy serverless propre (`api/search.js`) protégeant les clés d'API (TMDB, OMDb) et implémentant du cache HTTP (`s-maxage`).
- Stratégie hors-ligne (PWA, Service Worker).

Points faibles :

- **Couplage fort UI/État** : Le DOM est souvent utilisé comme source de vérité (ex: lectures directes de `.dataset` ou `classList.contains`), et les mutations du DOM sont mélangées avec la logique de manipulation de données.

- **Absence de système de composants** : Les templates HTML sont générés via des chaînes de caractères interpolées (`12-movie-detail.js`). Cela expose aux erreurs XSS (bien qu'atténué par `escAttr`) et rend l'évolution des UI difficile.
- **État global** : Les données (Historique, Watchlist) sont constamment lues et réécrites intégralement dans le `localStorage` (`loadHistory()` , `saveHistory()`), avec des risques de désynchronisation ou de pertes lors d'opérations concurrentes.
- **Extensibilité** : Ajouter une nouvelle fonctionnalité implique souvent de modifier plusieurs fichiers et de câbler manuellement des événements globaux sur le `document` .

Qualité du Code

Score : 6/10

Points forts :

- Code relativement propre, documenté avec des commentaires très explicites expliquant le *pourquoi* des choix techniques.
- Bon naming global des variables et fonctions.
- Pas de dépendances inutiles.

Points faibles :

- **Magic Strings et Nombres** : Présents notamment dans les requêtes et la gestion du DOM.
- **Listeners globaux** : Écouteurs délégués attachés à `document` ou `body` de manière tentaculaire (ex: la gestion des modales, ou des boutons). Cela crée une complexité cognitive lors du debugging.
- **God Objects / Fonctions trop longues** : Par exemple, `buildMdsContent` dans `12-movie-detail.js` construit un HTML complexe d'une traite.
- **Gestion des erreurs** : Basique. Les `catch` font souvent un `return` ou affichent un état d'erreur en dur dans le DOM, mais manquent parfois de granularité.

Fonctionnalités

Score : 9/10

L'application est très complète pour son périmètre.

Points forts (Premium) :

- Système de notation granulaire (7 critères) très pensé.
- Statistiques riches (Heatmap, radar, "Wrapped" annuel).
- Gamification (Système Elo pour les duels de films, badges).
- Synchro Cloud intelligente (fusion asynchrone, tombstones pour les suppressions).

Points d'amélioration :

- La recherche manque peut-être d'un état de chargement plus réactif.
- Les listes très longues ("Classiques à explorer") pourraient bénéficier d'une vraie virtualisation, le `content-visibility` CSS est une bonne astuce mais montre ses limites.

UX

Score : 8/10

Points forts :

- Navigation rapide, feedback immédiat.
- Le système d'onglets unifié et les gestes de balayage (swipe) rendent l'utilisation mobile naturelle.
- Les modales "Bottom sheet" s'ouvrent de manière fluide avec la possibilité de fermer par balayage vers le bas.

Points faibles :

- La gestion de la touche retour du navigateur (`20-back-navigation.js`) avec un seul état fictif peut parfois surprendre l'utilisateur.

UI

Score : 9/10

L'interface donne une impression "Premium", particulièrement grâce à :

- Un excellent usage des polices et de l'échelle typographique.
- Les multiples thèmes cohérents qui modifient l'ambiance sans casser la structure (ex: le verre dépoli du thème "Moderne", le filtre sépia du thème "Cinéphile").
- La composition en grilles (nouveau dans Ludex 2.0).
- Les couleurs d'accent calculées dynamiquement d'après les affiches.

Fluidité & Animations

Score : 9/10

Points forts :

- Système de mouvement bien pensé (`--dur-fast` , `--dur-base` , `--dur-slow` et `--ease-out` en CSS).
- Micro-interactions haptiques artificielles soignées (`.tap-pop` , animations SVG personnalisées pour chaque icône).
- Squelettes de chargement (skeleton loaders) pertinents.

Performance

Score : 8.5/10

- **Frontend** : Bundle très léger (Vanilla JS, pas de framework). Images chargées en lazy loading.
- **Backend** : Excellent usage des en-têtes `Cache-Control` (`s-maxage` , `stale-while-revalidate`) pour soulager les appels TMDb.
- **Limites** : Le parsing massif du DOM par `innerHTML` pour de grandes listes peut engendrer des "layout thrashing" (reflows coûteux) sur des appareils bas de gamme. L'absence d'une vraie virtualisation (virtual scrolling) JS est la seule ombre au tableau.

Accessibilité

Score : 8/10

- **Points forts** : Focus visible géré explicitement (`:focus-visible`), respect du `prefers-reduced-motion` , cibles cliquables de taille convenable.
- **Faiblesses** : L'usage de `div` avec `role="button"` et `tabindex="0"` est répandu. Bien qu'un écouteur global pallie le manque de support natif pour `Enter/Space` , l'utilisation d'éléments natifs `<button>` devrait être privilégiée pour éviter des failles ARIA.

Responsive

Score : 8/10

La bascule Mobile/Desktop est bien maîtrisée avec une grille CSS qui s'adapte via `auto-fill / minmax`. La disparition du multi-colonnes CSS problématique de l'historique sur desktop est une excellente correction.

Sécurité

Score : 7/10

- **Points forts** : Le backend Vercel valide correctement les formats (id numériques) avant l'interpolation d'URL. Le `sync_code` est désormais haché côté serveur (`sha256`) et exigé dans le Header `X-Sync-Code`.
- **Points faibles** : Le frontend utilise massivement `innerHTML`. Une fonction `escAttr()` est utilisée, mais la moindre omission dans un nouveau composant pourrait introduire une faille XSS persistante. Une approche de rendu sécurisée ou l'utilisation de `textContent` / `createElement` serait plus sûre.

Tests

Score : 7.5/10

- Les fonctions logiques critiques (`03b-pure-logic.js`) sont testées (Node).
- La présence annoncée de tests End-to-End (Playwright) est excellente.
- L'absence de tests unitaires sur les composants UI est normale vu l'architecture Vanilla, mais rend les régressions visuelles possibles en dehors de la couverture Playwright.

Maintenabilité

Score : 5/10

C'est le point noir de l'application.

- L'architecture par concaténation de fichiers empêche l'utilisation de modules ES6 standards, limitant l'encapsulation (tout est quasi global) et le Tree Shaking.
- Le mélange Vue/Logique dans de gros blocs de Template Strings rend l'évolution difficile pour de nouveaux développeurs.
- Le système de build est artisanal (`build-app-js.js`).

Design System

Score : 8.5/10

Un véritable Design System "Headless" est présent dans le CSS :

- Variables d'espacement (`--space-*`)
- Typographie hiérarchique
- Système d'ombres cohérent
- Theming implémenté via des attributs data (`data-theme`).

Dette technique

P1 — CRITIQUE : Risque XSS potentiel dû à l'utilisation généralisée d' `innerHTML` . **P2 — IMPORTANT** : Architecture globale de build (`app.js` généré par concaténation) et état global géré par `localStorage` muté à de nombreux endroits. **P3 — AMÉLIORATION** : Remplacement des `div role="button"` par des balises `<button>` .

Top 20 des problèmes

Problème	Gravité	Impact	Effort	Priorité
1. Sécurité XSS : Utilisation intensive de <code>innerHTML</code> pour générer l'UI.	CRITIQUE	Fort	Moyen	P1
2. Architecture : Concaténation des scripts plutôt que l'usage de modules ES6 (ESM).	ÉLEVÉE	Fort	Important	P2
3. État partagé : Les mutations du <code>localStorage</code> sont dispersées.	ÉLEVÉE	Moyen	Important	P2
4. God Objects : Fonctions UI monolithiques (ex: <code>buildMdsContent</code>).	MOYENNE	Moyen	Important	P3
5. Accessibilité : Usage de <code>div role="button"</code> au lieu de <code>button</code> .	MOYENNE	Faible	Faible	P2
6. Performance : Pas de virtualisation JS pour l'Historique massif.	MOYENNE	Faible	Moyen	P3
7. Couplage Fort : Logique métier non-pure intimement liée au DOM (hors 03b).	MOYENNE	Moyen	Important	P3
8. Gestion des Événements : Trop d'écouteurs délégués sur <code>document</code> .	MOYENNE	Faible	Moyen	P3
9. Sécurité Backend : Rate limiting en mémoire volatile (non partagé entre instances Vercel).	FAIBLE	Faible	Moyen	P3
10. Robustesse Backend : Fallbacks parfois silencieux cachant de vraies erreurs TMDb.	FAIBLE	Faible	Faible	P3
11. CSS : Répétitions malgré les variables (ex: animations définies de multiples fois).	FAIBLE	Faible	Faible	P3
12. Build : Le script de minification est rudimentaire.	FAIBLE	Faible	Faible	P3
13. UI : Marge de sécurité pour éviter le chevauchement du texte sur certains petits écrans.	FAIBLE	Faible	Faible	P3
14. Architecture : <code>discoverLoaded</code> state géré globalement dans la navigation.	FAIBLE	Faible	Faible	P3

15. Testing : Pas de tests unitaires sur les helpers du DOM.	FAIBLE	Faible	Important	P3
16. UX : Comportement du bouton retour natif (20-back-navigation.js) simulé.	FAIBLE	Moyen	Moyen	P3
17. Typage : Absence de TypeScript ou JSDoc limitant l'autocomplétion et la sécurité.	FAIBLE	Fort	Important	P3
18. CSS : Usage de !important sur les transitions globales.	FAIBLE	Faible	Faible	P3
19. UX : Squelettes de chargement parfois liés au délai réseau plutôt qu'à l'état de la donnée.	FAIBLE	Faible	Moyen	P3
20. Code Mort : Fonctions abandonnées non nettoyées dans certains anciens fichiers.	FAIBLE	Faible	Faible	P3

Quick Wins

- Remplacer systématiquement les `<div role="button">` par des vrais `<button>` avec reset CSS.
- Ajouter des Headers de sécurité stricts (CSP) via Vercel pour limiter le risque XSS inhérent à l'`innerHTML`.
- Migrer le `Rate Limiting` en mémoire de Vercel vers Vercel KV (Redis) pour un vrai respect des limites.

Plan de refactoring & Roadmap d'amélioration

Phase 1 — Corrections critiques (Sécurité & Accessibilité)

- **Objectifs** : Sécuriser le DOM et finaliser le support WCAG.
- **Tâches** : Implémenter des Content Security Policies (CSP), remplacer `div role="button"` par `<button>`, s'assurer que `escAttr` est appelé à 100% des cas d'interpolation de données externes.
- **Difficulté** : Moyenne.
- **Bénéfice** : Réduction drastique de la surface d'attaque.

Phase 2 — Performance & Architecture de build

- **Objectifs** : Passer à un bundler moderne (Vite).
- **Tâches** : Remplacer le script de concaténation par Vite, migrer les fichiers en modules ES (`import / export`), configurer le Tree Shaking.
- **Difficulté** : Importante.
- **Bénéfice** : Dev Experience (HMR), encapsulation stricte, disparition des variables globales.

Phase 3 — Architecture de l'état (State Management)

- **Objectifs** : Isoler les lectures/écritures du LocalStorage.
- **Tâches** : Créer une classe/module "Store" centralisant la data. Le DOM doit s'abonner aux changements du Store plutôt que de relancer `renderAll()` massivement.
- **Difficulté** : Importante.
- **Bénéfice** : Fin des bugs de synchronisation, performances accrues.

Phase 4 — Refactoring UI / Templates

- **Objectifs** : Supprimer l'usage brut de `innerHTML` .
- **Tâches** : Introduire une librairie légère de templating (comme `lit-html` ou un VDOM basique) pour lier le DOM et l'état de manière sûre et réactive.
- **Difficulté** : Très Importante.
- **Bénéfice** : Sécurité XSS garantie, code plus propre et maintenable.

Phase 5 — TypeScript & Qualité

- **Objectifs** : Sécuriser les structures de données.
- **Tâches** : Migrer de `.js` vers `.ts` (ou ajouter la JSDoc stricte).
- **Difficulté** : Moyenne.
- **Bénéfice** : Documentation intégrée, moins de bugs de runtime.

Score final

Domaine	Score	Justification
Architecture	6/10	Architecture Vanilla datée (concaténation), mais avec une bonne séparation de la pure logique.
Qualité du code	6/10	Code lisible mais pénalisé par l'état global et les fonctions monolithiques.
Fonctionnalités	9/10	Très complet, riche, bien pensé pour le "Power User".
UX	8/10	Navigation fluide, mais gestion du "Back" perfectible.
UI	9/10	Très belle direction artistique et thèmes soignés.
Fluidité	9/10	Design tokens d'animation excellents.
Responsive	8/10	Bien géré via CSS Grid.
Sécurité	7/10	Backend sécurisé, Frontend exposé via <code>innerHTML</code> .
Tests	7.5/10	Tests backend et logique pure présents, E2E avec Playwright.
Maintenabilité	5/10	Le manque de modules ES et de bundler rend le projet fragile à grande échelle.
Design System	8.5/10	Beau système de design headless via CSS variables.
NOTE GLOBALE	7.5/10	Un projet ambitieux, très beau et complet fonctionnellement, mais dont l'architecture "Vanilla" artisanale atteint ses limites pour un passage à l'échelle.

Conclusion

Ludex Rating Companion est l'œuvre d'un ingénieur produit ayant une forte sensibilité design et UX. Le produit fini rivalise avec des applications de niveau professionnel en termes d'apparence et de richesse de fonctionnalités. Cependant, l'approche technique choisie (Vanilla JS sans module loader, génération massive de DOM par des chaînes de caractères, état global non structuré) constitue une dette technique importante. Si l'application doit grandir, intégrer de nouveaux développeurs ou

supporter une refonte, la priorité absolue sera de migrer la base de code vers des standards modernes de l'écosystème JS (Vite, Modules ES, State Manager dédié, et potentiellement TypeScript), sans perdre l'excellente identité visuelle et les performances actuelles.